Object-Oriented Programming

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)

COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY

Asabhya Maanav

2026 EDITION

VERSION 1.0

Table of Contents

Object-Oriented Programming (Java) — One-Liners

Rapid-revision, double-layered one-liners for the **IBPS SO (IT Officer)** exam. Every line pairs a **structural fact** (how many / which / when / what size) with the **named items**. Critical keywords, numbers and code render in red.

OOP Concepts & Fundamentals

1. There are **4** pillars of OOP: **Abstraction**, **Encapsulation**, **Inheritance**, and **Polymorphism**.
2. The syllabus compares **2** paradigms: **procedural** (process-oriented) and **object-oriented** (object-centric).
3. There are **2** design directions: OOP is **bottom-up** while procedural programming is **top-down**.
4. A class and an object differ in **1** key way: a class is a **blueprint/template**, while an object is its **runtime instance** that occupies memory.
5. The 4-pillar mnemonic is **A-PIE**: **Abstraction**, **Polymorphism**, **Inheritance**, **Encapsulation**.
6. **Message passing** in OOP happens through exactly one mechanism: **method calls/invocations** between objects.
7. Beyond the 4 pillars there are **2** supporting concepts: **class/object** and **message passing**.
8. There are **2** core object relationships: **IS-A** (inheritance) and **HAS-A** (composition/aggregation).
9. Procedural code organizes around **functions** (e.g. in **C**), while OOP organizes around **objects** (e.g. in **Java**).

Java Basics — Introduction & Architecture

10. Java was created by **1** lead designer, **James Gosling**, at **Sun Microsystems**.
11. Java had **1** original name, **Oak**, renamed to **Java** in **1995**.
12. Java's project began in **1991**, with its first kit **JDK 1.0** released in **1996**.
13. Java's slogan is **WORA**, expanded as **Write Once, Run Anywhere**.
14. Sun Microsystems was acquired by **1** company, **Oracle**, in **2010**.
15. Java compiles source to **1** intermediate form, **bytecode**, stored in `.class` files.
16. There are **2** key tools: `javac` (compiler, `.java` to `.class`) and `java` (launcher running on the **JVM**).
17. There are **3** layered components: **JDK** (develop), **JRE** (run), and **JVM** (execute).
18. There are **2** containment rules: **JDK = JRE + tools** and **JRE = JVM + libraries**.
19. There is **1** portability twist: Java **bytecode** is platform independent but the **JVM** is platform dependent.
20. The entry method has **1** fixed signature: `public static void main(String[] args)`.
21. A source file may hold many classes but only **1 public** class, whose name must match the file name.
22. Java has about **50** reserved keywords, including `goto` and `const` which are reserved but unused.
23. Java has **3** reserved literals that are not keywords: `true`, `false`, and `null`.
24. Java uses **2** execution stages: it is **compiled** to bytecode and then **interpreted** by the JVM.
25. The JVM internally has key parts: **class loader**, **bytecode verifier**, **interpreter**, and **JIT compiler**.

Data Types, Variables & Type Casting

26. Java has **8** primitive types: **byte**, **short**, **int**, **long**, **float**, **double**, **char**, **boolean**.
27. **byte** occupies **1 byte (8 bits)** with range **-128 to 127**.
28. **short** occupies **2 bytes (16 bits)** with range **-32768 to 32767**.
29. **int** occupies **4 bytes (32 bits)** and is the default **integer** type.
30. **long** occupies **8 bytes (64 bits)** and uses the suffix **L**.
31. **float** occupies **4 bytes (32 bits)** and uses the suffix **f**.
32. **double** occupies **8 bytes (64 bits)** and is the default **floating-point** type.
33. **char** occupies **2 bytes (16 bits)** in Java, holding **Unicode** values **0 to 65535**.
34. **boolean** stores only **2** values, **true** and **false**, with **no precisely defined size**.
35. Default values number **4** kinds: **0** for numerics, **\u0000** for char, **false** for boolean, **null** for objects.
36. There are **8** wrapper classes: **Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, **Character**, **Boolean**.
37. There are **3** variable kinds by scope: **local** (stack), **instance** (heap), and **static** (method area).
38. **Local** variables have exactly **0** default values, so they must be explicitly initialized.

39. Integer caching covers the range **-128 to 127** for the `Integer.valueOf` method.

40. There are **2** conversions: **widening** (implicit, safe) and **narrowing** (explicit, lossy).

41. The widening order has **6** steps: **byte to short to int to long to float to double**.

42. There is **1** special widening: **char** auto-widens to **int** but needs a cast for byte/short.

43. **Narrowing** requires exactly **1** thing: an explicit **cast** in parentheses.

Operators & Expressions

44. Java has **1** ternary operator, `?:`, which takes **3** operands.

45. There are **2** short-circuit logical operators: `&&` (AND) and `||` (OR).

46. There are **3** shift operators: `<<` (left), `>>` (signed right), `>>>` (unsigned right).

47. There is **1** modulus operator, `%`, returning the **remainder**.

48. There are **4** bitwise operators: `&`, `|`, `^`, and `~`.

49. There are **2** binding rules: **precedence** (which first) and **associativity** (tie-breaker direction).

50. There is **1** right-to-left group worth noting: **assignment** and **ternary** operators.

51. The `+` operator has **2** roles: numeric **addition** and **String concatenation**.

Control Statements

52. There are **3** categories of control statements: **selection**, **iteration**, and **jump**.

53. There are **4** loop constructs: **for**, **while**, **do-while**, and **for-each** (enhanced for).

54. The **do-while** loop runs at least **1** time because the condition is checked **after** the body.

55. There are **3** jump statements: **break**, **continue**, and **return**.

56. **switch** accepts these types: **byte**, **short**, **int**, **char**, **String** (Java 7+), and **enum**.

57. **switch** rejects **4** types: **long**, **float**, **double**, and **boolean**.

58. A missing **break** in switch causes **1** behaviour: **fall-through** to following cases.

Arrays

59. Array indexing is **0-based**, ranging from **0 to length-1**.

60. Array size is read via the `length` field, which is **1** field, not a method.

61. There is **1** array type with uneven rows: the **jagged array**.

62. An invalid index throws **1** unchecked exception: `ArrayIndexOutOfBoundsException`.

63. Arrays have **2** traits: they are **objects** on the heap and are **fixed-size** after creation.

64. A 2D array `int[3][4]` holds **12** elements arranged as **3** rows by **4** columns.

Classes, Objects & Constructors

65. The `new` operator does **2** things: allocates on the **heap** and returns a **reference**.

66. A constructor has **2** rules: **same name as the class** and **no return type**.

67. There are **3** constructor kinds: **default**, **no-argument**, and **parameterized**.

68. The compiler adds **1** constructor automatically only when **0** constructors are written.

69. Java provides **0** built-in copy constructors, unlike **C++**.

70. **Constructor overloading** means **2 or more** constructors with **different parameter lists**.

71. The `this()` call must appear in exactly **1** position: the **first statement** of a constructor.

72. Constructors have **3** restrictions: they cannot be **final**, **static**, or **abstract**, and are not inherited.

Methods, Parameter Passing & Recursion

73. **Method overloading** changes **1** thing — the **parameter list** — and is resolved at **compile-time**.

74. There is **1** thing that cannot distinguish overloaded methods: the **return type** alone.

75. Java uses exactly **1** parameter-passing mechanism: **pass-by-value** (the **reference value** is copied for objects).

76. **Recursion** needs **1** mandatory element, a **base case**, to avoid `StackOverflowError`.

77. Each method call adds **1** **stack frame** to the call stack.

Keywords, Access Control & Members

78. There are **2** self/parent references: `this` (current object) and `super` (immediate parent).

79. A **static** member has exactly **1** shared copy for the whole class.

80. A **static block** runs exactly **1** time, when the class is **loaded**, before `main`.

81. **Static** methods have **2** limits: they cannot use `this`/`super` or access instance members directly.

82. The `final` keyword has **3** uses: **constant variable**, **non-overridable method**, **non-extendable class**.

83. **String** and all **8** wrapper classes are declared **final**.

84. There are **3** classic confusables: `final` (constant), `finally` (always-run block), `finalize()` (GC cleanup).

85. Java has **4** access modifiers: **private**, **default**, **protected**, **public**.

86. Access visibility widens in **4** steps: **private to default to protected to public**.

87. **private** members have visibility limited to **1** scope: the **same class**.

88. **protected** members are visible in **2** scopes: the **same package** and **subclasses** elsewhere.

89. **default** (no modifier) gives **1** scope of access: **package-private**.

90. A top-level class allows only **2** modifiers: **public** or **default**.

Garbage Collection

91. **Garbage collection** reclaims memory of objects with exactly **0** live references.

92. `System.gc()` does **1** thing only: it **requests** GC without guaranteeing it.

93. `finalize()` was called **1** time by the GC before reclaiming an object (deprecated in **Java 9**).

94. Java has **0** manual memory operators: there is no `delete`/`free` and no **destructor**.

Nested Classes & String Handling

95. There are **4** nested-class types: **static nested**, **inner**, **local**, and **anonymous**.

96. A non-static **inner** class can access all outer members, including **private** ones.

97. An **anonymous inner class** has **0** names and is common for **listeners** and **Runnable**.

98. A **static nested** class needs **0** outer instances to be created.

99. **String** objects have **1** key property: they are **immutable**.

100. String literals live in **1** special area: the **String Constant Pool (SCP)** in the heap.

101. `new String("x")` always creates **1** new heap object, bypassing the pool.

102. There are **2** comparison rules: `==` compares references, `equals()` compares content.

103. There are **2** mutable string classes: **StringBuffer** (synchronized) and **StringBuilder** (not synchronized).

104. **StringBuilder** was introduced in **Java 5** and is **faster** than StringBuffer.

105. There are **3** size members: `length()` (String), `length` (array), `size()` (collection).

Inheritance

106. Inheritance uses **1** keyword, `extends`, modelling an **IS-A** relationship.

107. Java directly supports **3** inheritance types: **single**, **multilevel**, and **hierarchical**.

108. Java supports **0** multiple inheritance of classes to avoid the **diamond problem**.

109. There is exactly **1** root class for all Java classes: `java.lang.Object`.

110. `super()` calls the parent constructor and must be the **first** statement of the subclass constructor.

111. Constructor chaining executes in **1** order: **parent-first (top-down)**.

112. **Method overriding** keeps the signature the **same** and is resolved at **runtime**.

113. **Covariant return types** allow an overriding method to return a **subtype**.

114. There are **3** methods that cannot be overridden: **private**, **static**, and **final** methods.

Polymorphism

115. There are **2** polymorphism types: **compile-time** (overloading) and **runtime** (overriding).

116. **Compile-time** polymorphism uses **1** binding type: **static/early binding**.

117. **Runtime** polymorphism uses **1** mechanism: **dynamic method dispatch (late binding)**.

118. In overriding, the executed method depends on **1** thing: the **object type**, not the reference type.

119. **Upcasting** assigns a subclass object to **1** kind of reference: a **superclass** reference.

Encapsulation & Abstraction

120. **Encapsulation** combines **2** things — data and methods — using **private fields + public getters/setters**.

121. **Abstraction** hides implementation via **2** tools: **abstract classes** and **interfaces**.

122. An **abstract class** allows **0** instantiations but can still have **1 constructor**.

123. A class needs only **1** abstract method to be forced **abstract** itself.

124. An **abstract method** has **0** body and must be **overridden** by the first concrete subclass.

Packages

125. A **package** serves **1** main purpose: a **namespace** that avoids name conflicts.

126. The `package` statement must be in **1** position: the **first** statement of a file.

127. There is **1** auto-imported package in every program: `java.lang`.

128. **CLASSPATH** answers **1** question: **where** to find classes and JAR files.

129. Key built-in packages include `java.lang`, `java.util`, `java.io`, `java.net`, `java.awt`, `javax.swing`.

130. There are **2** import forms: single-class `import pkg.Class` and on-demand `import pkg.*`.

Interfaces

131. Interface methods are implicitly **public abstract** and variables are implicitly **public static final**.

132. A class uses **1** keyword, `implements`, and can implement **many** interfaces.

133. Interfaces give Java **1** form of multiple inheritance: inheritance **of type**.

134. **Java 8** added **2** method kinds to interfaces: **default** and **static** methods.

135. **Java 9** added **1** more method kind to interfaces: **private** methods.

136. A **marker interface** has **0** methods; examples are **Serializable** and **Cloneable**.

137. A **functional interface** has exactly **1** abstract method; examples are **Runnable** and **Comparable**.

138. There are **2** abstraction containers: **abstract class** (single inheritance) and **interface** (multiple).

139. An interface gives **100%** abstraction (pre-Java 8) while an abstract class gives **partial** abstraction.

Exception Handling

140. The exception hierarchy has **1** root: `java.lang.Throwable`.

141. **Throwable** has **2** direct branches: **Error** and **Exception**.

142. There are **2** exception categories: **checked** (compile-time) and **unchecked** (runtime).

143. **Unchecked** exceptions share **1** superclass: **RuntimeException**.

144. Checked examples include **IOException**, **SQLException**, **ClassNotFoundException**, and **InterruptedException**.

145. Unchecked examples include **NullPointerException**, **ArithmeticException**, **ArrayIndexOutOfBoundsException**, and **NumberFormatException**.

146. **Error** examples include **OutOfMemoryError** and **StackOverflowError**, which should not be handled.

147. The **finally** block executes in all cases except **1**: a call to `System.exit()`.

148. There are **2** keywords: `throw` (throws **1** instance) and `throws` (declares possibly **many** types).

149. **Multi-catch** (Java 7) joins types with **1** symbol, the pipe `|`.

150. In multiple catch, the **most specific** exception must come **before** the more general one.

151. **try-with-resources** (Java 7) auto-closes resources implementing **1** interface: **AutoCloseable**.

152. A **custom exception** extends **1** of two classes: **Exception** (checked) or **RuntimeException** (unchecked).

153. Integer division by zero throws **1** exception: **ArithmeticException**.

Multithreading

154. A **thread** is the **smallest** unit of execution within a **process**.
155. The `Thread.State` enum has **6** states: **NEW**, **RUNNABLE**, **BLOCKED**, **WAITING**, **TIMED_WAITING**, **TERMINATED**.
156. There are **2** ways to create a thread: **extend Thread** or **implement Runnable**.
157. Implementing **Runnable** has **1** key advantage: the class can still **extend** another class.
158. To launch a thread you call **1** method, `start()`, never `run()` directly.
159. Calling `run()` directly creates **0** new threads, running in the **current** thread.
160. `sleep()` is a **static** method that releases **0** locks while pausing.
161. There is **1** difference: `wait()` releases the lock while `sleep()` does not.
162. `join()` makes **1** thread wait for **another** to finish.
163. `yield()` hints the scheduler to favour **equal-priority** threads.
164. Thread priority ranges **1 to 10**: **MIN_PRIORITY=1**, **NORM_PRIORITY=5**, **MAX_PRIORITY=10**.
165. The default thread priority is **5**, also called **NORM_PRIORITY**.
166. **Synchronization** lets exactly **1** thread hold an object's **monitor/lock** at a time.
167. There are **2** synchronization forms: **synchronized method** and **synchronized block**.
168. `wait()`, `notify()`, and `notifyAll()` are defined in **1** class: `Object`, not Thread.
169. There are **2** wake methods: `notify()` wakes **1** thread, `notifyAll()` wakes **all**.
170. Calling wait/notify outside a synchronized context throws **1** exception: **IllegalMonitorStateException**.
171. **Deadlock** arises from **1** condition: **circular waiting** on mutually held locks.

Event Handling

172. The **Delegation Event Model** has **3** participants: **source**, **event object**, and **listener**.
173. Event classes reside in **1** package: `java.awt.event`.
174. A button click generates **1** event, **ActionEvent**, handled by **ActionListener**.
175. **MouseEvent** is handled by **2** interfaces: **MouseListener** and **MouseMotionListener**.
176. **KeyEvent** is handled by **KeyListener**, and **WindowEvent** by **WindowListener**.
177. **Adapter classes** provide **empty** implementations of listeners that have **many** methods.
178. **ActionListener** has **0** adapter classes because it declares only **1** method.
179. Listeners are registered through **1** style of method: `addXxxListener()`.

Applets, AWT & Swing

180. An applet has **0** `main()` methods and runs inside a **browser** or **appletviewer**.
181. The applet life cycle has **5** methods: **init**, **start**, **paint**, **stop**, **destroy**.
182. Among life-cycle methods, **2** run once — **init()** and **destroy()** — while **start()/stop()** run many times.
183. The `paint(Graphics g)` method has **1** job: **drawing** the applet output.
184. There are **2** applet base classes: `java.applet.Applet` (AWT) and `javax.swing.JApplet` (Swing).
185. Applets were **deprecated** in **Java 9** and **removed** in **Java 17**.
186. There are **2** GUI toolkits: **AWT** (heavyweight, native peers) and **Swing** (lightweight, pure Java).
187. AWT lives in `java.awt` while Swing lives in `javax.swing`.
188. Swing components share **1** prefix, the letter **"J"** (JButton, JLabel, JFrame).
189. **BorderLayout** has **5** regions: **North**, **South**, **East**, **West**, **Center**.
190. Default layouts differ: a **Frame/Window** uses **BorderLayout**, a **Panel/Applet** uses **FlowLayout**.
191. Common layout managers number **6**: **FlowLayout**, **BorderLayout**, **GridLayout**, **GridBagLayout**, **CardLayout**, **BoxLayout**.
192. Swing supports **1** extra feature AWT lacks: a **pluggable look-and-feel**.

I/O Streams

193. Java I/O has **2** stream families: **byte streams** and **character streams**.

194. Byte streams have **2** roots — **InputStream** and **OutputStream** — for **binary** data.

195. Character streams have **2** roots — **Reader** and **Writer** — for **16-bit Unicode** text.

196. There are **3** standard streams: `System.in` (input), `System.out` and `System.err` (output).

197. There are **2** common console-input classes: **BufferedReader** and **Scanner**.

Java Distinctives & Extra Keywords

198. Java supports **0** operator overloading, unlike **C++**.

199. Java has **0** pointers and **0** global variables, improving **security**.

200. There are **2** unused reserved keywords in Java: `goto` and `const`.

201. The `instanceof` operator does **1** thing: tests an object's **type** at runtime.

202. **Autoboxing/unboxing** (Java 5) converts between **2** forms: a **primitive** and its **wrapper**.

203. The `transient` keyword excludes a field from **1** process: **serialization**.

204. The `volatile` keyword forces reads/writes from **1** location: **main memory**, not a thread cache.

205. The `strictfp` keyword enforces **1** guarantee: **platform-independent** floating-point results.

206. There are **2** keywords that cannot combine: `abstract` and `final` on the same class/method.

207. An **enum** represents **1** thing: a fixed set of named **constants**.

208. The `Object` class declares key methods: **equals()**, **hashCode()**, **toString()**, **getClass()**, **clone()**, and **finalize()**.

209. Classic textbooks list **5** thread states: **New**, **Runnable/Ready**, **Running**, **Blocked/Waiting**, and **Dead**.

210. There are **2** ways to achieve runtime polymorphism prerequisites: **inheritance** plus **method overriding**.